Intro

Learning Objectives

- Explain the differences between frontend applications using JavaScript with and without React
- Use **Vite** to set up a react app for local development
- Explain that a react component
  - couples together logic and presentation
  - is a JS function that returns JSX that represents part of the page
  - can store its own data.
- Build and export a simple react component and then import it appropriately using **ES6 Modules**

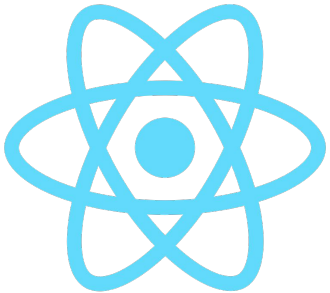
Intro

Vanilla JavaScript can be a **lot of fun**, but there are some aspects that add **friction** and make applications **hard to build** and **maintain**

Some of the common **pain points** are:

- How the UI is **created**
- How functionality is **split** up across the app
- How **data** is stored and managed
- How the UI is **updated**

React aims to solve these problems - let's take a look at how!



JS: How the UI is created

Creating the user interface in JavaScript can get tedious really fast

We either have to create every element by hand using **document.createElement**, or we have to use the risky **innerHTML** that may allow users to inject code into our app!

It's definitely not as easy as the good old days of writing **plain HTML**

React: How the UI is created

React solves this problem by introducing new syntax into JS: **JSX**

Here's a basic React component:

```
function Title () {
 return <h1 className="title">Welcome to React!</h1>
}
```

Q: Notice anything familiar?

React: How the UI is created

```
function Title () {
 return <h1 className="title">Welcome to React!</h1>
}
```

That's right! It looks like our **Title** function is just returning **HTML**, but how is that possible!?

This HTML-looking syntax is actually **JSX***. Behind the scenes, this code is being transformed into React's version of **createElement** for us, but our development experience has now become much more seamless!

* The **X** in **JSX** stands for XML, which stands for Extensible Markup Language.
XML is similar to HTML, but without predefined tags to use. Instead, you define your own tags designed specifically for your needs.
It will become clear as we learn more about React why this is.

Read more about XML here

JS: How functionality is split

There is no one way, and that's the problem

With so much **freedom**, you will find that no two codebases are **alike**. Some might put everything in a **single file**, others might **split files** based on differing criteria, a single **function** might do **very little**, or **build the entire application**

Q: What part takes care of state? What takes care of rendering?

React: How functionality is split

This is one of the places where React really shines

```
function Title () {
  return <h1 className="title">Welcome to React!</h1>
}
```

React is opinionated: it asks you to think and reason about your user interface in **components**.

A single **component** is responsible for part of the page. It can **receive data**, **store data**, and describe what to **render** on the page for that part.

JS: How data is stored

Again, the freedom is the problem here

By default, a lot of the time you will end up storing your data **in the DOM**, which is a terrible way to do it, since **JS is so much better** at handling data

Our **JS state** approach makes things a lot easier, and this is actually very close to how **React** manages things!

React: How data is stored

```
function Title () {
 const [name, setName] = useState("Booleaners")
 return <h1 className="title">Welcome to React, {name}!</h1>
}
```

In **React**, every **component** has the ability to **store and manage its own data**

JS: How the UI is updated

We pretty much have to go and **manage the DOM ourselves** here, which is unnecessarily complex

React: How the UI is updated

React manages the UI on our behalf!

Forget having to **querySelect**, **append**, **innerText**, etc ever again!

Any time you **change your app's state**, React will go and **update the page** where needed to reflect those changes

In a nutshell

What	JS	React
Creating UI	**createElement**	**JSX**
Organising the functionality	**Many ways**	**Components**
Storing data	**Many ways**	**Component state**
Updating UI	**Manual**	**Automatic**

React brings **convention** and a **standard API** to the table when building UIs.

There is a **learning curve** to React, but you will soon find out how much **fun** it can be!

Time for some live coding

Let’s look at our first React component in action!

JSX

As you just saw, **JSX** gives us a way for us to **write HTML within our JS**:

```
export default function App () {
 return (
   <div className="app">
     <h1 className="title">Welcome to React!</h1>
     <p>Things are about to get fun!</p>
   </div>
 )
}
```

Let's see what's actually going on though

Q: What's the one different thing between this code and HTML? Why is that?

Starting up

```
import ReactDOM from 'react-dom'
import App from './App'

const root = ReactDOM.createRoot(document.getElementById('root'))
root.render(<App />)
```

To get our app running, we need the following **index.js** file

Let's explore what is going on line by line

Starting up

```
import ReactDOM from 'react-dom'
import App from './App'

const root = ReactDOM.createRoot(document.getElementById('root'))
root.render(<App />)
```

First, we need to **import ReactDOM**, using **ES6 Modules**
Importing is how we **bring code from libraries** or **other files** so that we can use it in the current file

Starting up

```
import ReactDOM from 'react-dom'
import App from './App'

const root = ReactDOM.createRoot(document.getElementById('root'))
root.render(<App />)
```

Next, we **import** the **App** component we created **from** our **App.js** file

Q: What's different between the ReactDOM import and this one?

Starting up

```
import ReactDOM from 'react-dom'
import App from './App'

const root = ReactDOM.createRoot(document.getElementById('root'))
root.render(<App />)
```

Now we need to **getElementById** the root element from our **index.html** file

This is the only time we will ever need to query the DOM manually again, because we will give this to React, and it will **manage its contents** for us

Starting up

```
import ReactDOM from 'react-dom'
import App from './App'

const root = ReactDOM.createRoot(document.getElementById('root'))
root.render(<App />)
```

Finally, we call our **ReactDOM.render** function to render our component

This function is where the magic happens and our app gets added to the page

Vite

Here's how we can set up a react app for local development.

In a terminal, run the following command:

$ npm create vite@latest my-react-app -- --template react

Vite is a "build tool" which creates and runs web apps using React extremely quickly and easily.

See here for official documentation

Vite

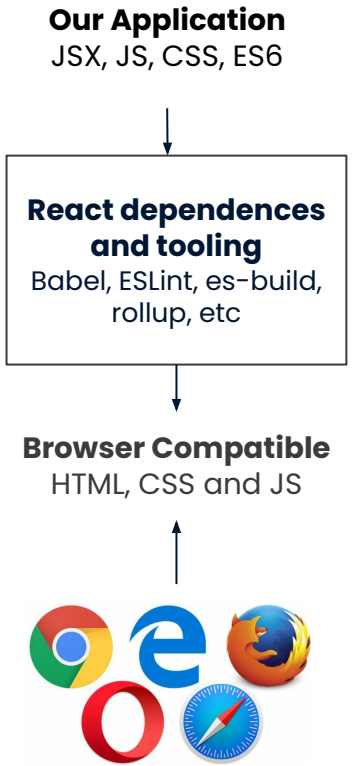
What just happened?

- npm create is an alias for npm install - it installs the Vite package, and then runs a command to "scaffold" the project for you.
- This scaffolding involves:
  - Creating a project directory with a basic directory structure and placeholder files
  - Creating a package.json file
  - Installs React dependencies

React Dependencies and Tooling

React uses features and tools which browsers don't support (JSX) and some features supported only by newer Browsers (ES6 Modules).

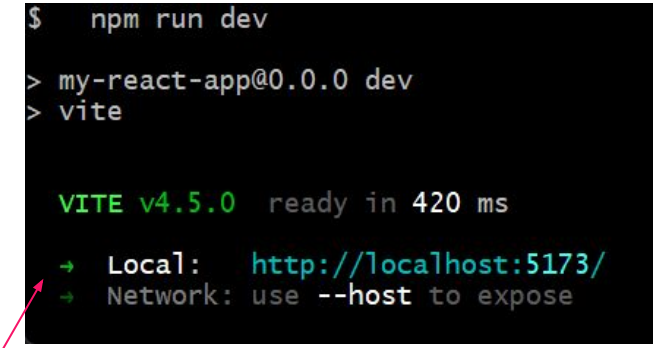
Vite sets up tooling that **makes our application runnable by the browser**, and also provides features that make it easier to develop, package and distribute our application.



Vite

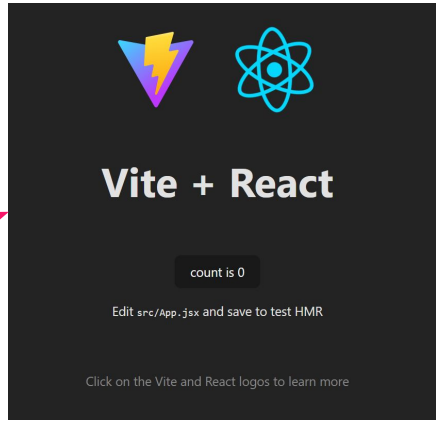
Let's fire it up!

```
$ cd my-react-app
$ npm i
$ npm run dev
```



Open a browser and go to the specified localhost address (e.g. http://localhost:5173/)

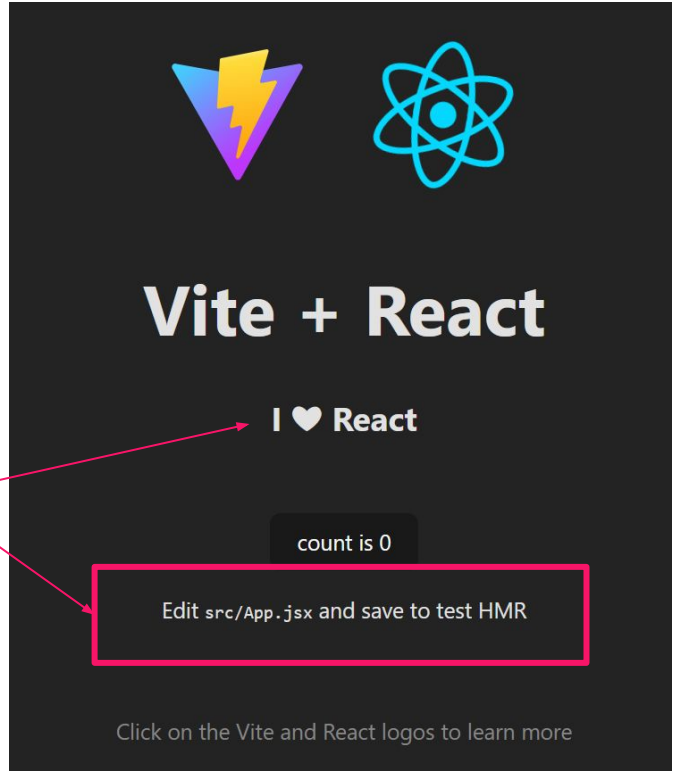
It should open this page:



Vite

Hot Module Replacement (HMR)!
Make a change to the file src/App.jsx, save it and see it in your browser!

No more unnecessary browser refreshing



Time for some live coding

Let’s create a simple React app using vite!

Wait! Let's check our Node.js version...

The local development server that Create-React-App provided for us uses node. For it to work correctly you need to be using the current stable version of node.js. The *very* latest version of node may not work. In your command line (*GitBash*, for Windows users) type:

- **nvm list**

If you see anything with v16 at the front you are good! If you see v17 or v18, then take the following steps:

- Close any VS Code windows you have open
- Open a new terminal (not in VSCode). If you are on Windows, you'll need to run the terminal as an Administrator (Right-click, select Run As Administrator)
- Run the command **nvm install 16**
- This will install the current version of node
- On windows, once installed the command will show you the **nvm use** command to use that version - run it. For Mac/Linux, you may need to run: **nvm alias default 16**

Time for some practice

- **Create a React template using vite:**

```
$ npm create vite@latest my-react-app -- --template react
$ cd my-react-app
$ npm i
$ npm run dev
```

NOT: `npm ci`

- **Remove all of the default code in `src/App.jsx` until you have an empty function**
- **In your function, return some elements using standard HTML tags/structure**
- **Give your elements classes (using the `className` attribute, NOT the `class` attribute)**
- **Remove all of the default template CSS within `src/App.css`**
- **Add styles for your elements**

Interpolation

```
function App () {
 const name = "Booleaners"
 return (
   <div className="app">
     <h1 className="title">Welcome to React, {name}!</h1>
     <p>Things are about to get fun!</p>
   </div>
 )
}
```

Another great thing about JSX is that we can **interpolate any JS expression** using **{}**

Q: What will the output of this code be?

Interpolation

```
function App () {
 const name = "Booleaners"
 return (
   <div className="app">
     <h1 className="title">Welcome to React, {name.toUpperCase()}!</h1>
     <p>Things are about to get fun!</p>
   </div>
 )
}
```

Q: And this?

Nesting components

```
// App.js
function App () {
 return (
   <div className="app">
     <h1 className="title">Welcome to React!</h1>
     <p>Things are about to get fun!</p>
   </div>
 )
}
```

So, this is our first component, and it's pretty neat! We could just write our **entire app** in this single component, but it would grow **out of control** rather quickly

To solve this, React lets us create **more components** and **nest them** within our JSX

Refactoring Strategy: Extract Component

Q: Sound familiar?

```
// App.js
function App () {
 return (
   <div className="app">
     <h1 className="title">Welcome to React!</h1>
     <p>Things are about to get fun!</p>
   </div>
 )
}
```

For example, we can take this **title** and turn it into its own component

Q: How do you think we might do that?

Extracting components

```
// App.js
import Title from './Title.js'

function App () {
 return (
   <div className="app">
     <Title />
     <p>Things are about to get fun!</p>
   </div>
 )
}
```

```
// Title.js
function Title () {
 return <h1 className='title'>Welcome to React!</h1>
}

export default Title
```

For example, we can take this **title** and turn it into its own component

Q: How do you think we might do that?

Extracting components

```
// App.js
import Title from './Title.js'

function App () {
 return (
   <div className="app">
     <Title />
     <p>Things are about to get fun!</p>
   </div>
 )
}
```

```
// Title.js
function Title () {
 return <h1 className='title'>Welcome to React!</h1>
}

export default Title
```

There you go! All we had to do was create a new function that returns that HTML, export it, import it in your App and then add it as a tag right next to the rest of our HTML

Q: What else could we extract? How?

Extracting components

```
// App.js
import Title from './Title.js'
import Text from './Text.js'

function App () {
 return (
   <div className='app'>
     <Title />
     <Text />
   </div>
 )
}
```

```
// Text.js
function Text () {
 return <p>Things are about to get fun!</p>
}

export default Text
```

Yup, just as easily, we now extracted our **Text** into its own component, too

Divide and conquer

```
// App.js
import Title from './Title.js'
import Text from './Text.js'

function App () {
 return (
   <div className='app'>
     <Title />
     <Text />
   </div>
 )
}
```

Hopefully you can see how this starts to simplify our code

The game of React is about getting good at modelling the User Interface as **layers** of **components**, rather than just a **big HTML blob**

Divide and conquer

```
function Text () {
 return <p>Things are about to get fun!</p>
}
```

Each **component** on its own is more **cohesive** and we can still create a **large application** by combining lots of components. This is the same idea you've seen in the module Development Approaches using JS Classes - in fact you'll see **modularity** repeatedly in every facet of software development

Time for some live coding

Let’s see that in action!

Import vs Require

In **server-side** JavaScript (**NodeJS**), we used CommonJS **module.exports/require**.
In **browser-side** JavaScript (**React**), we will use ES6 Modules **export/import**.

```
// App.js
import Title from './Title.js'

function App () {
 return (
   <div className="app">
     <Title />
     <p>Things are about to get fun!</p>
   </div>
 )
}
```

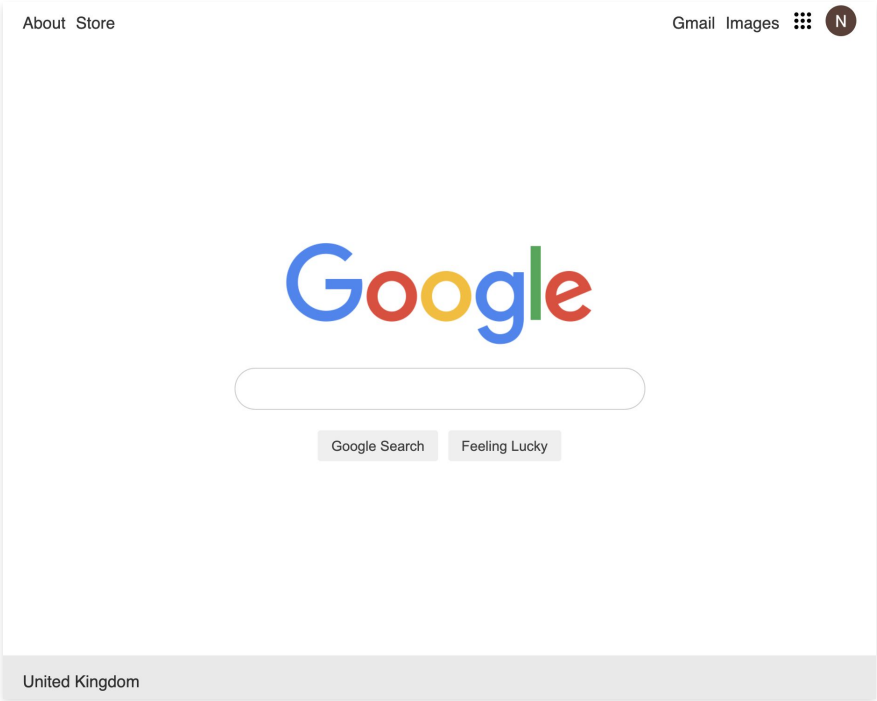
```
// Title.js
function Title () {
 return <h1 className='title'>Welcome to React!</h1>
}

export default Title
```

This works in a similar way to **module.exports** from node. We need it so other files can "see" our components.

Divide and conquer

Q: How would you divide this page into components?



Divide and conquer

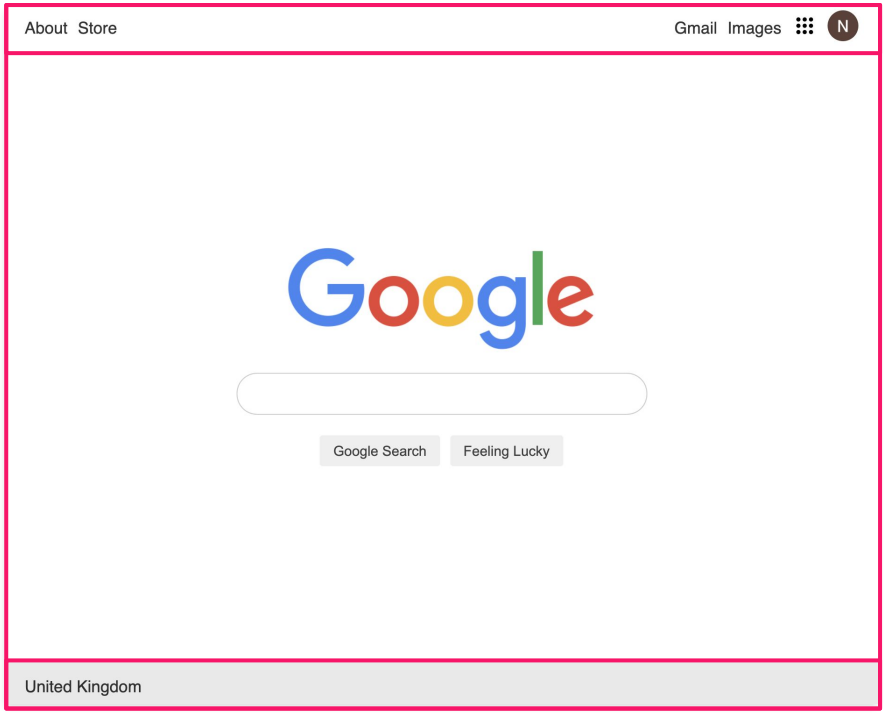
<Header />

<Main />

A: Here's one way

Q: Can we take it even further?

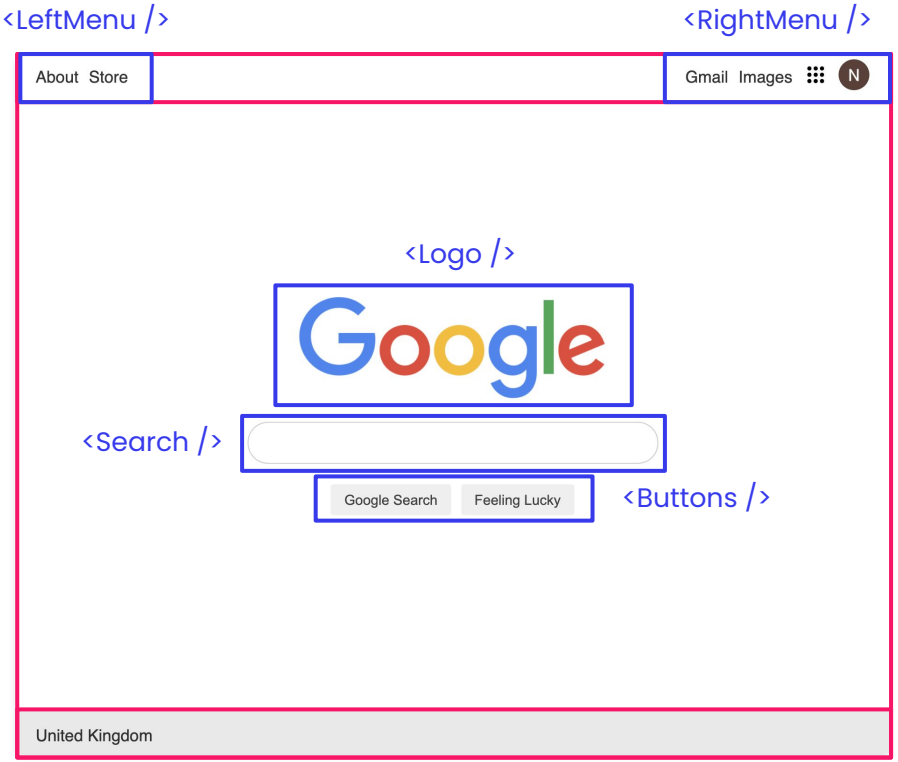
<Footer />



Divide and conquer

A: Of course we can!

Q: Anything else? ... You get the point 🙂



Time for some practice

Let's make those changes

https://github.com/boolean-uk/react-components-intro